

SECTION 7: Miscellaneous Topics

(I may add additional topics in the future.)

Lecture 24: Timing Your Code

(and dynamic memory/allocatable arrays)

You may want to try and optimize your code for speed. An easy way to time the relevant sections of the code is very useful.

We will use the `system_clock` function to do this.

In general, add to your code before and after the sections you wish to time:

`CALL SYSTEM_CLOCK(count, countrate, countmax)`

All the arguments are integers, must be of the same level precision, and are optional. In particular:

`count` is increased by one for each “clock count”

`countrate` returns the number of counts/second (depends on precision)

`countmax` sets the maximum value that `count` can have

Hence units of `count/countrate` are in seconds.

Lets try an example with a Laplace solver. We will look at the influence of the compiler optimization switch on run time.

```

!PROGRAM TO CHECK CODE RUN TIME
!USE DIFFERENT OPTIMIZATION SWITCHES
!USE ALLOCATABLE ARRAYS
MODULE VARIABLES
IMPLICIT NONE
SAVE
INTEGER :: nx, ny, iterations
REAL, ALLOCATABLE, DIMENSION(:, :) :: phi, phiold
END MODULE VARIABLES

!*****

!MAIN PROGRAM
PROGRAM LAPLACE_SOLVER
USE VARIABLES
IMPLICIT NONE
INTEGER :: status, l, i, j, &
           count_start, count_end, count_rate
REAL :: wall_time

PRINT*
WRITE(*,*) 'Enter number of iterations: '
READ(*,*) iterations
PRINT*
WRITE(*,*) 'Enter the number of cells in one direction: '
READ(*,*) nx;  ny=nx
ALLOCATE (phi(0:nx+1,0:ny+1), phiold(0:nx+1,0:ny+1), stat=status)
IF(status /= 0) THEN
    WRITE(*,*) 'Error in allocating memory.'

```

```

        STOP
    END IF
!initialize variables
phi=0; phi(0,:)=100.; phi(:,0)=100.; phiold=phi;

CALL SYSTEM_CLOCK(count_start,count_rate)    !start timing

DO l=1,iterations
    phi(1:nx,1:ny)=0.25*(phi(2:nx+1,1:ny) + phi(0:nx-1,1:ny) + &
        phi(1:nx,0:ny-1) + phi(1:nx,2:ny+1))
END DO

CALL SYSTEM_CLOCK(count_end)        !end timing
wall_time=real(count_end-count_start)/real(count_rate)
PRINT*
PRINT*, 'Run Time = ', wall_time
PRINT*
IF(nx .le. 5) then
    do j=ny+1,0,-1
        write(*,100) (phi(i,j),i=0,nx+1)
    end do
ELSE
    PRINT*, '(Array too large to write out on screen.)'
END IF
100    FORMAT(7f10.2)
DEALLOCATE(phi,phiold)
END PROGRAM LAPLACE_SOLVER

```

```
G:\FORTRAN_COURSE\SECTION7-MISC\CODES2>g95 laplace.f90
G:\FORTRAN_COURSE\SECTION7-MISC\CODES2>a.exe
Enter number of iterations:
10000
Enter the number of cells in one direction:
100
Run Time = 1.0756
(Array too large to write out on screen.)
G:\FORTRAN_COURSE\SECTION7-MISC\CODES2>g95 -O2 laplace.f90
G:\FORTRAN_COURSE\SECTION7-MISC\CODES2>a.exe
Enter number of iterations:
10000
Enter the number of cells in one direction:
100
Run Time = 0.2601
(Array too large to write out on screen.)
G:\FORTRAN_COURSE\SECTION7-MISC\CODES2>
```

Lecture 25: DISLIN Graphics Package Overview

What is DISLIN? An excerpt from the DISLIN website:

“DISLIN is a high level library of subroutines and functions that display data graphically. It is intended to be a powerful and easy to use software package for programmers and scientists that does not require knowledge of hardware features. The software is available for the programming languages C/C++, Fortran 77 and Fortran 90/95.” DISLIN is free for non-commercial use.

Download DISLIN from:

<https://www.mps.mpg.de/dislin>

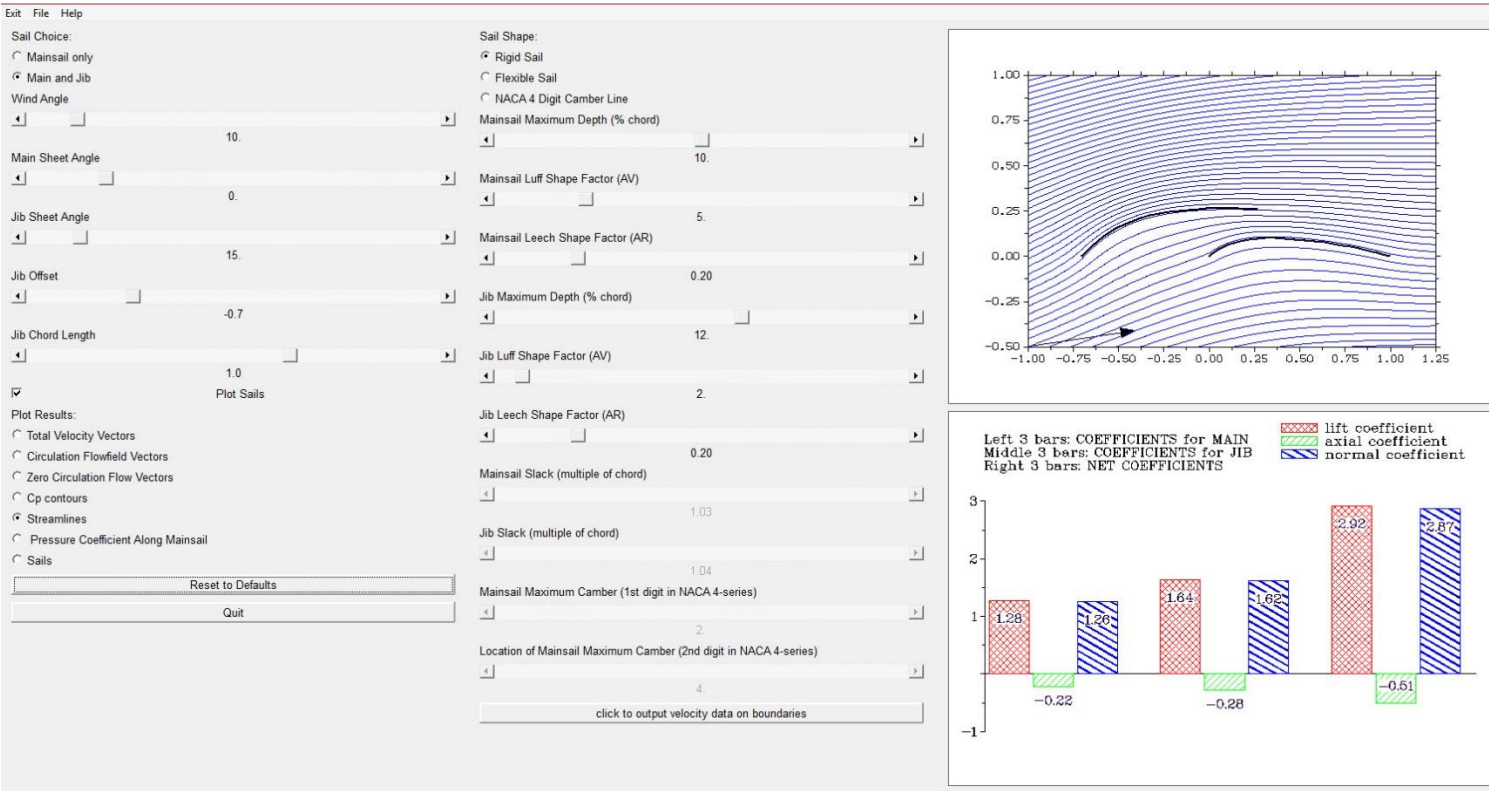
If you are running Windows, I recommend downloading the 32 bit version for Mingw. The file name is “d11_mg.zip”

See example DISLIN widgets, including one by author (where the code is all Fortran) at the link:

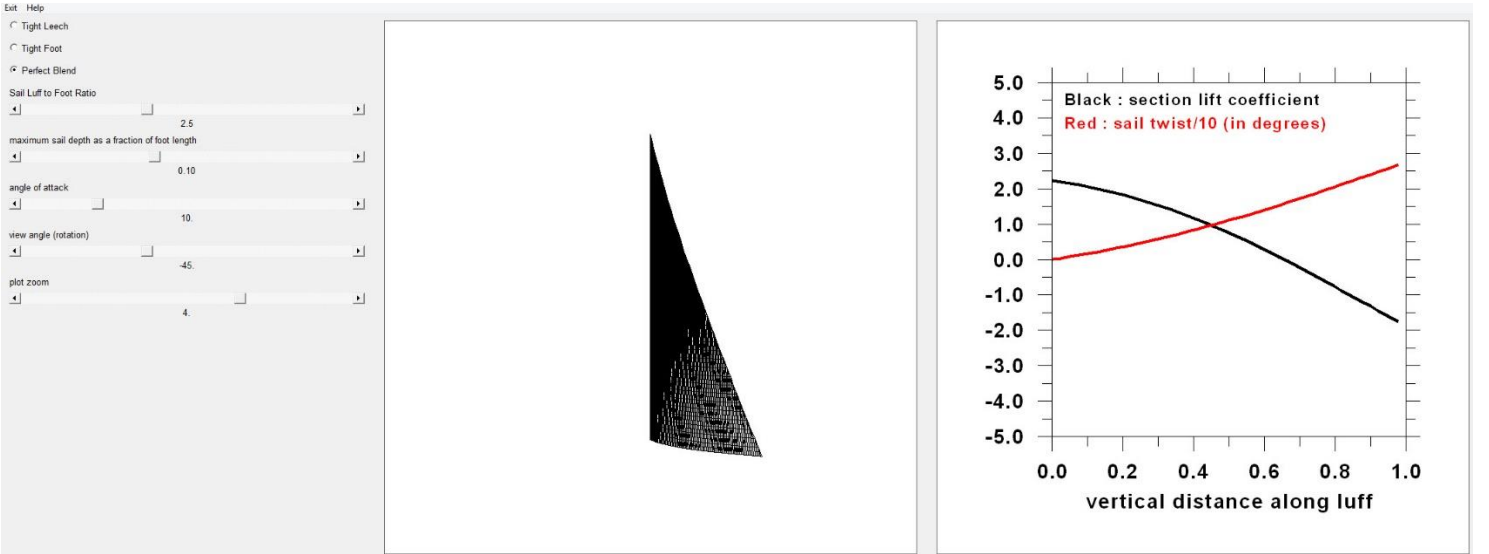
https://www.mps.mpg.de/1758995/gallery_widgets2#sailanalysis

Example DISLIN Screens From My Fortran Codes

2D Potential Flow Solver



Sailboat Sail Shape Analysis



Simple Sailboat Velocity Prediction Program (VPP)

ExitTerms and ConditionsHelp

wind speed in knots

10.

wind direction relative to boat centerline

45.

main sheet angle relative to boat centerline

17.0

sail area (m^2)

25.

sail aspect ratio

4.0

sail Oswald efficiency factor

0.8

sail zero lift parastitic drag coeff

0.08

keel area (m^2)

1.5

keel aspect ratio

2.0

keel Oswald efficiency factor

0.8

keel zero lift parastitic drag coeff

0.08

push to reset to default values

Actual Wind

Apparent Wind

Boat Heading (leeway angle)

Boat Centerline

Total Force Vector (scaled)

Aerodynamic Lift and Drag Forces

Driving and Heeling Forces

leeway angle (deg.)	boat speed (knots)	velocity made good	driving force (N)	lateral force (N)	total force (N)
3.49	4.17	1.42	301.29	660.37	725.86

sail lift coefficient	sail drag coefficient	keel lift coefficient	keel drag coefficient
1.043	0.188	0.191	0.087

apparent wind speed (knots)	apparent wind angle (degrees)	sail angle of attack (degrees)
13.1	34.8	14.3

7

Lecture 26: DISLIN Example 1

```
! USING A QUICK PLOT ROUTINE
! EXAMPLE OF USING DISLIN TO MAKE LINE PLOTS
USE DISLIN
IMPLICIT NONE
INTEGER, PARAMETER :: NPTS=100
REAL, PARAMETER :: PI=3.141526
REAL, DIMENSION(1:NPTS) :: x, y, y1
INTEGER :: i, ioutput
CHARACTER(20):: file_name
! set up data points
DO i=1,NPTS
    x(i)=(REAL(i-1)/REAL(NPTS-1))*2.*PI
    y(i)=SIN(x(i))      !data for sin function
    y1(i)=COS(x(i))     !data for cosine function
END DO
PRINT*
PRINT*, 'Enter 1 for screen output, 2 for .svg file output'
READ(*,*)ioutput
PRINT*
PRINT*, 'For screen output, right click mouse button to advance
plots'

IF(ioutput .eq. 1)then
    CALL METAFL('XWIN')      !set output to screen
    CALL SCRMOD("REVERS") !reverse to a black line on white
background
    CALL DISINI      !initialize DISLIN
```



```

CALL NAME('x (radians)', 'X')    !label x axis
CALL NAME('sin(x)', 'Y')         !label y axis
CALL QPLOT(x,y,NPTS)            !call quick plot, opens in a window
PRINT*, 'right click mouse button to advance plot'
CALL QPLCRV(x,y,NPTS, 'FIRST')   !both sin and cos on
CALL QPLCRV(x,y1,NPTS, 'LAST')   !one plot
ELSE IF(ioutput .eq. 2)then      !output plot to file instead
    CALL METAFL('SVG')           !scalable vector graphics format
    PRINT*, 'Enter the 1st output graph file name (w/o
extension):'
    READ(*, '(A)')file_name;
    CALL SETFIL(TRIM(file_name)//'.svg')
    CALL SCRMOD("REVERS") !reverse to a black on white background
    CALL DISINI
    CALL NAME('x (radians)', 'X')
    CALL NAME('sin(x)', 'Y')
    CALL QPLOT(x,y,NPTS)
    PRINT*, 'Enter the 2nd output graph file name'
    READ(*, '(A)')file_name;
    CALL SETFIL(TRIM(file_name)//'.svg')
    CALL DISINI
    CALL NAME('x (radians)', 'X')
    CALL NAME('sin(x) or cos(x)', 'Y')
    CALL QPLCRV(x,y,NPTS, 'FIRST')
    CALL QPLCRV(x,y1,NPTS, 'LAST')
ELSE
    PRINT*, 'You did not enter a 1 or 2'
END IF;  END

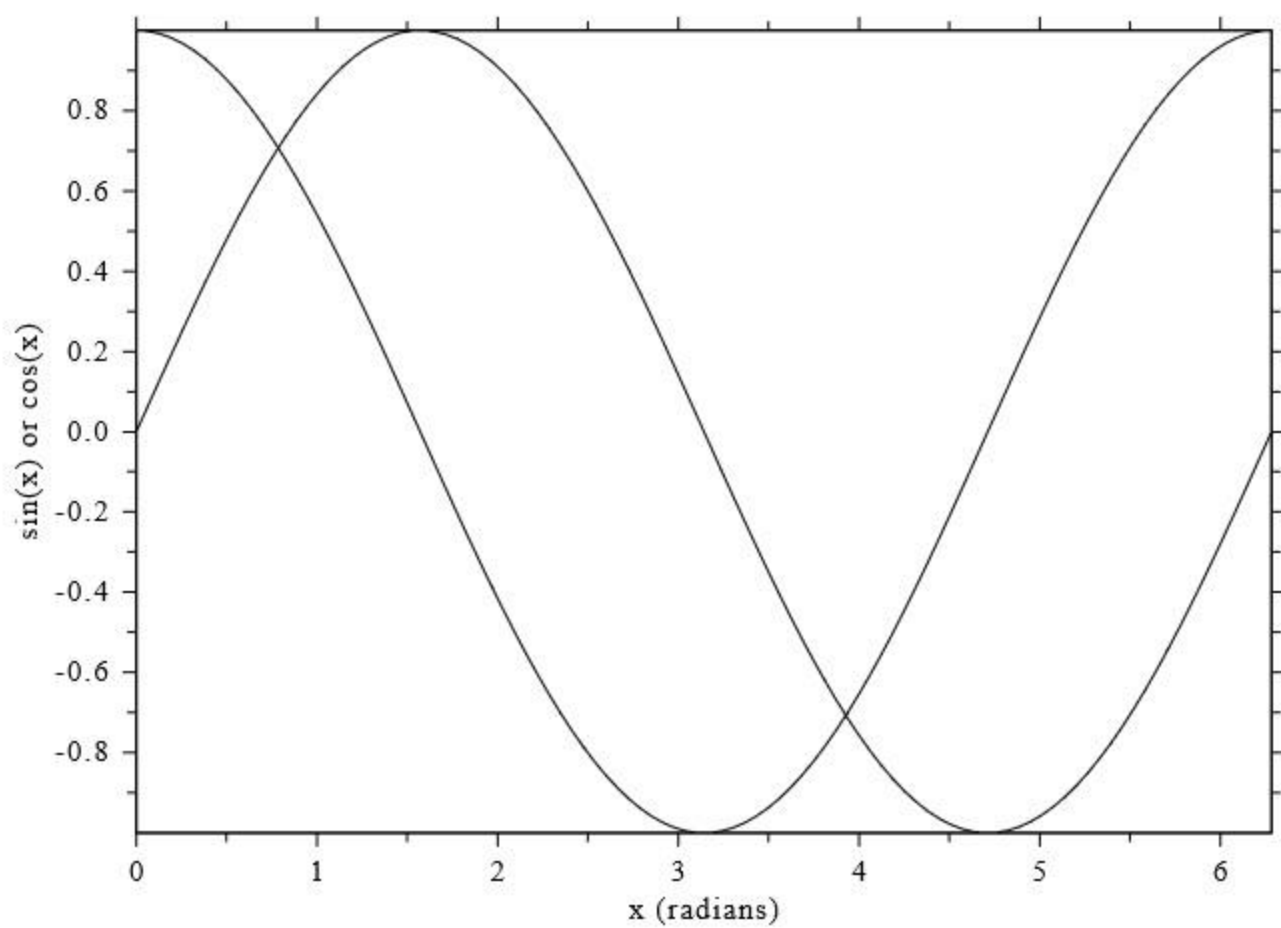
```

```
gfortran -O2 -fdefault-real-8 -c -Ic:\dislin\gf\real64
lineplot.f90

gfortran -o lineplot.exe lineplot.o -static c:\dislin\disgf_d.a
-luser32 -lgdi32 -lopengl32

lineplot.exe
```





A print of one of the .svg files.

Lecture 27: DISLIN Example 2

```
MODULE variables
  IMPLICIT NONE
  SAVE
  REAL::pi,r,cablelength,mass,g=9.81,tanalpha,alpha,depth,&
    rho,mu,cd,v,w,re,towdepth
  INTEGER :: isize,jsize,ip,ipl,ipa,id,id_r, &
    id_mass,id_cablelength,id_v,id_cd,id_box1,id_defaults
END MODULE variables

PROGRAM TOWED_SPHERE
  USE DISLIN
  USE VARIABLES
  IMPLICIT NONE
  INTEGER :: nxp,nyp
  CHARACTER(1400) :: helptext
  EXTERNAL :: graphics, getdata, resetvars

  rho=1027.      ! density of sea water at 10 degrees C

  mu=0.00141     ! sea water viscosity at 10 degrees C

  ! the following text shows up in the help menu
  helptext='GENERAL INSTRUCTIONS||&
  &This code computes the depth at which an underwater|&
  &tow vehicle must travel to cause a towed sphere|&
```

```

&to submerge below the surface. Drag from tow line|&
&is neglected.||&
&User enters relevant data in the GUI including a|&
&reasonable drag coefficient based on Reynolds number.|&
&Results appear simultaneously in DOS window as input values|&
&are varied.'

```

```

CALL SWGTIT('Towed Sphere Analysis') !add title

```

```

pi=4.0*atan(1.0)

```

```

CALL GETSCR(isize,jsize)

```

```

CALL SCLMOD('FULL')

```

```

PRINT*

```

```

PRINT*, 'screen size: ', 'isize=', isize, ' jsize=', jsize

```

```

CALL swgwin(0,0,nint(0.25*isize),nint(0.4*jsize))

```

```

CALL METAFL('XWIN')      !sets output to screen

```

```

CALL SCRMOD("REVERS")    !black on white background

```

```

CALL PAGE(900,900)        !set page square so vector angles are
correct

```

```

CALL SWGHLP(helptext)     !set help menu

```

```

CALL WGINI('HORIZ',ip)    !create main widget with index ip

```

```

CALL SWGWTH(-20)           !sets default width for widgets

```

```

CALL WGBAS(ip,'VERT',ipl)  !container widget for input data

```

```

!   SET UP WIDGETS FOR INPUT DATA

CALL WGLAB(ipl,'sphere radius in meters',id)  !create widget
label

CALL SWGSTP(0.01)    !define step value for scale widget

CALL WGSCL(ipl,' ',0.01,0.5,0.18,2,id_r)  !create a scale widget

CALL GWGSCL(id_r,r)  !returns radius into variable r


CALL WGLAB(ipl,'sphere mass in kg',id)
CALL SWGSTP(0.1)
CALL WGSCL(ipl,' ',0.,60.,20.4,1,id_mass)
CALL GWGSCL(id_mass,mass)
w=9.81*mass    ! compute sphere weight in newtons


CALL SWGSTP(0.1)
CALL WGLAB(ipl,'cable length in meters',id)
CALL WGSCL(ipl,' ',0.,40.,18.3,1,id_cablelength)
CALL GWGSCL(id_cablelength,cablelength)


CALL WGLAB(ipl,'two vehicle speed in m/s',id)
CALL SWGSTP(0.1)
CALL WGSCL(ipl,' ',0.,5.,2.5,2,id_v)
CALL GWGSCL(id_v,v)


PRINT*, ' _____ '
PRINT*; PRINT*

!   compute Reynolds number about sphere
re=rho*(2.*r)*v/mu

```

```

WRITE(*,60) re
60  format('Reynolds number=',f9.0)

CALL WGLAB(ipl,'drag coefficient',id)
CALL SWGSTP(0.005)
CALL WGSCL(ipl,' ',0.,1.,0.075,3,id_cd)
CALL GWGSCL(id_cd,cd)

CALL wgpbut(ipl,'Reset to Defaults',id_defaults)

!  callback routines to check for user data changes
!  if a sliderbar is moved, the getdata subroutine is called
!  through the SWGCBK calls below
CALL SWGCBK(id_r,getdata)
CALL SWGCBK(id_mass,getdata)
CALL SWGCBK(id_cablelength,getdata)
CALL SWGCBK(id_v,getdata)
CALL SWGCBK(id_cd,getdata)
CALL SWGCBK(id_defaults,resetvars)

CALL SOLUTION

CALL WGFIN

END PROGRAM towed_sphere

!*****

```

```

! subroutine to reread scale widget data if user moves slide bar
SUBROUTINE GETDATA
USE VARIABLES
IMPLICIT NONE

!  read in the data from WGSCL CALLs

!  get sphere radius
CALL GWGSCL(id_r,r)

!  get sphere mass
CALL GWGSCL(id_mass,mass)
w=g*mass    ! sphere weight in newtons

!  get cable length
CALL GWGSCL(id_cablelength,cablelength)

!  get underwater tow vehicle speed
CALL GWGSCL(id_v,v)

PRINT*, '_____ '
PRINT*; PRINT*

!  compute Reynolds number so user can update drag coeff.
re=rho*(2.*r)*v/mu
WRITE(*,60) re
60  FORMAT('Reynolds number=',f9.0)

```



```

!  get drag coefficient
CALL GWGSCL(id_cd,cd)

CALL SOLUTION
RETURN
END

!*****

!  subroutine to reset variables back to default
SUBROUTINE RESETVARS
USE VARIABLES
IMPLICIT NONE

CALL SWGSCL(id_r,0.18)
CALL SWGSCL(id_mass,20.4)
CALL SWGSCL(id_cablelength,18.3)
CALL SWGSCL(id_v,2.5)
CALL SWGSCL(id_cd,0.075)

CALL GETDATA

RETURN
END

!*****

!  subroutine to compute the solution
SUBROUTINE SOLUTION
USE VARIABLES
IMPLICIT NONE

```

```

tanalpha=(4./3.*pi*r**3*rho*g - w)/(0.5*cd*rho*v**2*pi*r**2)
alpha=ATAN(tanalpha)
depth=cablelength*SIN(alpha)
WRITE(*,50)alpha*180./pi
50  FORMAT('Cable angle with respect to surface=',F6.1,'
degrees. ')
IF(depth < 0.0)depth=0.0
WRITE(*,100)depth
100  FORMAT('If tow vehicle is below',F6.2,' meters sphere
will submerge. ')
PRINT*
RETURN
END

!*****

```

Towed Sphere Analysis

Exit Help

sphere radius in meters

◀ 0.18 ▶

sphere mass in kg

◀ 20.4 ▶

cable length in meters

◀ 18.3 ▶

two vehicle speed in m/s

◀ 2.50 ▶

drag coefficient

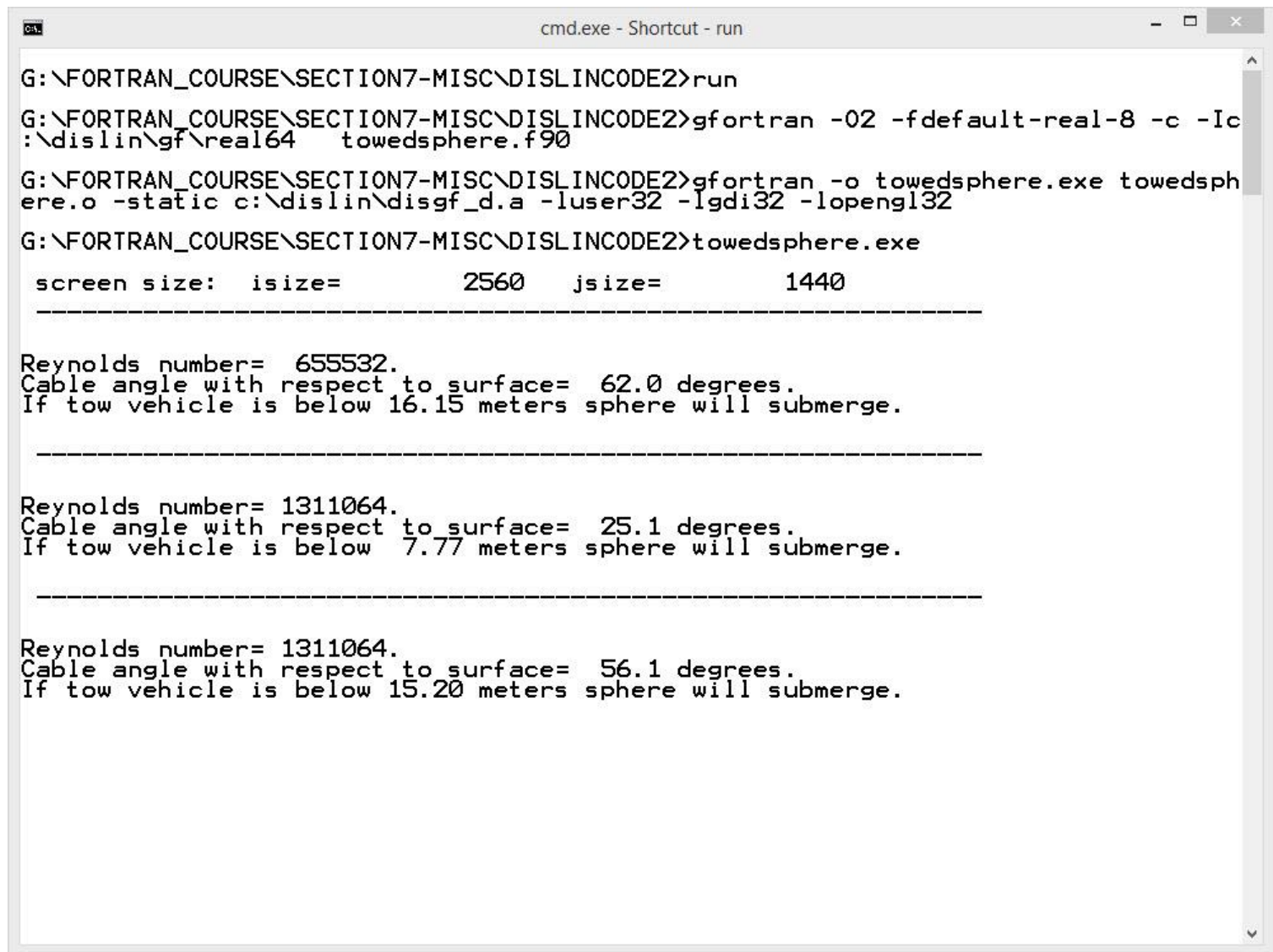
◀ 0.075 ▶

Reset to Defaults

OUTPUT BELOW IN DOS WINDOW

- 1) Output from default parameters above
- 2) Change speed to 5 m/s
- 3) Change sphere mass to 10.2 kg

Note I did not change drag coefficient after Reynolds number change. This was only for purposes of illustration. In practice, you should match the drag coefficient with the Reynolds number.



```
cmd.exe - Shortcut - run

G:\FORTRAN_COURSE\SECTION7-MISC\DISLINCODE2>run
G:\FORTRAN_COURSE\SECTION7-MISC\DISLINCODE2>gfortran -02 -fdefault-real-8 -c -Ic:\dislin\gf\real64 towedsphere.f90
G:\FORTRAN_COURSE\SECTION7-MISC\DISLINCODE2>gfortran -o towedsphere.exe towedsphere.o -static c:\dislin\disgf_d.a -luser32 -lgdi32 -lopengl32
G:\FORTRAN_COURSE\SECTION7-MISC\DISLINCODE2>towedsphere.exe

screen size:  isize=      2560  jsize=      1440
-----

Reynolds number= 655532.
Cable angle with respect to surface= 62.0 degrees.
If tow vehicle is below 16.15 meters sphere will submerge.

-----

Reynolds number= 1311064.
Cable angle with respect to surface= 25.1 degrees.
If tow vehicle is below 7.77 meters sphere will submerge.

-----

Reynolds number= 1311064.
Cable angle with respect to surface= 56.1 degrees.
If tow vehicle is below 15.20 meters sphere will submerge.
```